Задание

Цель работы: освоение базовых возможностей языка Python, работы со структурами данных, файловым вводом-выводом.

Реализовать на Python простейшую картотеку в виде консольного приложения в соответствии с вариантом согласованном в ЛР1.

Приложение включает:

1. Классы, описывающие элементы картотеки (предок/потом[ок|ки] — студент/староста/профорг, служащий/управляющий/директор и т.п.), содержащие атрибуты, описывающие элемент (имя, возраст и т.п.), поле/поля с ссылкой/ссылками на реализацию ввода/вывода, а также функцию либо оператор, возвращающую строковое представление объекта.

2. Класс, реализующий интерфейсную стратегию ввода/вывода для работы с консолью, функции которого в качестве параметра получают ссылку на объект и имя запрашиваемого поля в случае вывода.

3. Контейнерный класс (группа, компания и т.п.), содержащий:

— контейнер (список либо словарь) элементов;

— функцию добавления элемента (с выбором типа объекта);

— функцию вывода списка на экран;

— функцию чтения списка из файла;

— функцию записи списка в файл;

— функцию очистки списка.

4. Класс, реализующий стратегию ввода/вывода для работы с хранилищем на основе модуля pickle либо shelve, используемый контейнерным классом.

5. Функцию main, выводящую и обрабатывающую текстовое меню, которое позволяет воспользоваться всеми функциями, экспортируемыми контейнерным классом.

Требования к реализации:

— реализовать каждый класс в отдельном модуле, плюс один модуль

для функции, выводящей меню и запускающей функции контейнерного класса;

— взаимодействие с пользователем осуществляется через простое текстовое меню, выбор действия - ввод цифры;

— меню включает в себя следующие пункты: добавить объект выбранного типа, редактировать, удалить объект, вывести на экран весь список, сохранить в файл, загрузить из файла;

— реализовать обработку меню с помощью словаря/списка ссылок на функции.

Ход работы

1. Проектирование классовой структуры

Для реализации поставленной задачи была разработана следующая структура классов:

1.1 Классы элементов данных:

— Quest — описывает квесты. Содержит атрибуты: идентификатор, название, длительность (в минутах), минимальное и максимальное количество игроков. Данные о всех доступных квестах хранятся в глобальном списке all_quests.

— Booking — описывает бронирование квеста. Содержит атрибуты: идентификатор брони, идентификатор пользователя, идентификатор квеста, дата и время проведения, количество игроков.

— Review — описывает отзыв пользователя на квест. Содержит атрибуты: идентификатор отзыва, идентификатор пользователя, идентификатор квеста, оценки по трём критериям (общее впечатление, актёрская игра, логичность заданий) в диапазоне от 1 до 5, а также текстовый отзыв.

— User — описывает пользователя системы. Содержит атрибуты: идентификатор, имя, фамилия, номер телефона (с префиксом «+»), пароль, подтверждение пароля, электронная почта

(необязательно), время создания записи. Класс реализует методы валидации вводимых данных, методы ввода/вывода данных, а также методы редактирования профиля .

1.2 Контейнерный класс:

— Book — основной класс приложения. Выполняет роль контейнера, хранящего коллекции объектов: self.items (словарь) — пользователи системы; self.bookings (список) — бронирования; self.reviews (список) — отзывы; self.available_slots(список) — доступные временные слоты для бронирования.

— Реализует бизнес-логику: валидация данных, регистрация, авторизация, создание и отмену бронирований, добавление и редактирование отзывов.

— Содержит ссылки на стратегию хранения.

1.3 Класс стратегии хранения:

— PickleStorage — инкапсулирует логику работы с файловой системой. Использует модуль pickle для сериализации и десериализации объектов Python в бинарный файл delivery_data.dat.

— Методы: store() (сохранение), load() (загрузка).

2. Реализация интерфейса и меню

Взаимодействие с пользователем осуществляется через консоль. В функции main() реализовано главное меню, позволяющее выбрать режим работы (Админ-панель или Вход пользователя). Меню построено по принципу циклического опроса пользователя. Выбор действия обрабатывается через конструкцию ветвления, вызывающую соответствующие методы контейнерного класса Book.

Пункты меню включают:

— Создание/удаление/редактирование пользователей.

— Просмотр списка пользователей.

— Сохранение и загрузка состояния системы из файла.

— Личный кабинет пользователя (бронирование игр, оставление отзывов, отслеживание, профиль).

3. Валидация данных

Для обеспечения целостности данных в классе User реализован набор методов валидации:

— validate_name — проверка имени на наличие цифр, спецсимволов и минимальную длину (2 символа).

— validate_second_name — аналогичная проверка для фамилии.

— validate_phone — проверка номера телефона: 11 цифр, начинается с 7, отсутствие букв и спецсимволов.

— validate_password — проверка пароля: длина от 8 до 16 символов, наличие хотя бы одной латинской буквы.

— validate_email— проверка формата email: наличие символов «@» и «.», минимальная длина 5 символов. Поле не является обязательным.

При воде данных пользователем в методах read() и change() каждый ввод проходит через соответствующий валидатор; при неверном формате выводится сообщение об ошибке и запрос повторяется.

При вводе данных пользователем в методах read() и change()каждый ввод проходит через соответствующий валидатор; при неверном формате выводится сообщение об ошибке и запрос повторяется.

Листинг программы на языке программирования Python

1. Листинг файла book.py:

```
from dataclasses import dataclass
import pickle, datetime
from models import User, Booking, Review, all_quests
from storage import PickleStorage
from console import ConsoleIO
from models import all_quests

class Book:
    def __init__(self):
        self.items = dict()
        self.storage = PickleStorage(self)
        self.maxID = 0
        self.bookings = []
        self.available_slots = [
            "19.06.2025 15:30",
            "19.06.2025 17:00",
            "19.06.2025 20:00",
            "20.06.2025 15:30",
            "20.06.2025 17:00",
            "20.06.2025 20:00",
```

```
            "21.06.2025 15:30",
            "21.06.2025 17:00",
            "21.06.2025 20:00",
            "22.06.2025 15:30",
            "22.06.2025 17:00",
            "22.06.2025 20:00",
        ]
        self.reviews = []
        self.maxBookingID = 0
        self.maxReviewID = 0
        self.reviews = []
        self.maxBookingID = 0
        self.maxReviewID = 0
        self.current_user = None

    def add(self):
        self.maxID += 1
        item = User(id=self.maxID)
        item.read()
        self.items[item.id] = item

    def write(self):
        for id, item in self.items.items():
            item.write()

    def store(self):
        self.storage.store()

    def load(self):
        self.storage.load()

    def delete(self):
        id = int(input("Удаления пользователя с id: "))
        if id in self.items:
            del self.items[id]
            print("Пользователь с id ",id," был удален")
        else:
            print("Пользователь с таким id не найден")
    def change(self):
        id=int(input("Изменение пользователя с id: "))
        if id in self.items:
            self.items[id].change()
            print("Пользователь с id ", id, " был изменен")
        else:
            print("Пользователь с таким id не найден")
    def login(self):
        phone = input("Телефон (без +): ")
        password = input("Пароль: ")
        for user in self.items.values():
```

```
            if user.phone == "+" + phone and user.password ==
password:
                self.current_user = user
                print(f"Добро пожаловать, {user.first_name}!")
                return True
        print("Неверный телефон или пароль")
        return False
    def user_menu(self):
        while True:
            print("1. Редактировать профиль")
            print("2. Мои игры")
            print("3. Мои отзывы")
            print("4. Все квесты")
            print("5. Выйти из аккаунта")

            choice = input("Выбор: ")

            if choice == "1":
                self.current_user.change_profile()
            elif choice == "2":
                self.games()
            elif choice=="3":
                self.review()
            elif choice == "4":
                self.qwest()
            elif choice == "5":
                self.current_user = None
                print("Вы вышли")
                break
    def qwest(self):
            while True:
                print("Все квесты")
                print("1. Показать все квесты")
                print("2. Забронировать квест")
                print("3. Оставить отзыв")
                print("4. Назад")
                choice = input("Выбор: ")
                if choice == "1":
                    self.display_quests()
                elif choice == "2":
                    self.add_booking()
                elif choice == "3":
                    self.add_review()
                elif choice == "4":
                    break
                else:
                    print("Неверный выбор")
                    continue
    def display_quests(self):
```

```
        for quest in all_quests:
            print(f"{quest.id}. {quest.name}")
            print(f"Длительность: {quest.duration} мин")
            print(f"Игроки: от {quest.min_players} до
{quest.max_players}")
    def add_booking(self):
        while True:
            q= input("Введите номер квеста")
            if not q.isdigit():
                print("Введите число")
                continue
            q = int(q)
            if 1 <= q <= len(all_quests):
                quest = all_quests[q - 1]
                break
            else:
                print("Квест не найден")
        if not self.available_slots:
            print("Извините, нет доступных слотов для
бронирования")
            return
        print("Доступные слоты:")
        for i, slot in enumerate(self.available_slots, 1):
            print(f"{i}. {slot}")
        while True:
            slot_choice = input("Выберите слот: ")
            if not slot_choice.isdigit():
                print("Введите число")
                continue
            slot_choice = int(slot_choice)
            if 1 <= slot_choice <= len(self.available_slots):
                selected_slot = self.available_slots[slot_choice
- 1]
                break
            print("Неверный выбор")
        while True:
            player_count = input("Введите количество игроков: ")
            if player_count.isdigit():
                player_count = int(player_count)
                if quest.min_players <= player_count <=
quest.max_players:
                    break
                print("Неверное количество игроков")
            else:
                print("Введите число")
        self.maxBookingID += 1
        booking = Booking(
            id=self.maxBookingID,
            user_id=self.current_user.id,
```

```
            quest_id=quest.id,
            datetime=selected_slot,
            player_count=player_count
        )
        self.bookings.append(booking)
        self.available_slots.remove(selected_slot)
    def add_review(self):
        while True:
            q= input("Введите номер квеста")
            if not q.isdigit():
                print("Введите число")
                continue
            q = int(q)
            if 1 <= q <= len(all_quests):
                quest = all_quests[q - 1]
                break
            else:
                print("Квест не найден")
        for review in self.reviews:
            if review.user_id == self.current_user.id and
review.quest_id == quest.id:
                print("Вы уже оставляли отзыв на этот квест")
                return
        while True:
            over=input("Оцените общее впечатление от 1 до 5:")
            if not over.isdigit():
                print("Введите число")
                continue
            over=int(over)
            if 1 <= over <= 5:
                break
            print("Оценка должна быть от 1 до 5")
        while True:
            act = input("Оцените актерскую игру от 1 до 5:")
            if not act.isdigit():
                print("Введите число")
                continue
            act=int(act)
            if 1 <= act <= 5:
                break
            print("Оценка должна быть от 1 до 5")
        while True:
            logica = input("Оцените логику и связность заданий от
1 до 5:")
            if not logica.isdigit():
                print("Введите число")
                continue
            logica=int(logica)
            if 1 <= logica <= 5:
```

```
                break
            print("Оценка должна быть от 1 до 5")
        while True:
            text = input("Ваш отзыв: ")
            if len(text) < 10:
                print("Отзыв слишком короткий (минимум 10
символов)")
                continue
            elif len(text) > 500:
                print("Отзыв слишком длинный (максимум 500
символов)")
                continue
            else:
                break
        self.maxReviewID += 1
        review = Review(
            id=self.maxReviewID,
            user_id=self.current_user.id,
            quest_id=quest.id,
            overall=over,
            acting=act,
            logic=logica,
            text=text
        )
        self.reviews.append(review)
    def games(self):
        while True:
            print("Мои игры")
            print("1. Мои брони")
            print("2. Отменить бронь")
            print("3. Редактирование")
            print("4. Назад")
            choice = input("Выбор: ")
            if choice == "1":
                self.display_games()
            elif choice == "2":
                self.del_booking()
            elif choice == "3":
                self.change_booking()
            elif choice == "4":
                break
            else:
                print("Неверный выбор")
                continue
    def display_games(self):
        user_bookings = [b for b in self.bookings if b.user_id ==
self.current_user.id]
        if not user_bookings:
            print("У вас нет активных бронирований")
```

```
            return
        for i, booking in enumerate(user_bookings, 1):
            quest = None
            for q in all_quests:
                if q.id == booking.quest_id:
                    quest = q
                    break
            print(f"{i}. {quest.name}")
            print(f"   Номер брони: {booking.id}")
            print(f"   Дата и время: {booking.datetime}")
            print(f"   Количество игроков:
{booking.player_count}")
            print(f"   Длительность: {quest.duration} мин")
            print(f"   Игроков: {booking.player_count}")
    def del_booking(self):
        user_bookings = [b for b in self.bookings if b.user_id ==
self.current_user.id]
        if not user_bookings:
            print("У вас нет активных бронирований")
            return
        while True:
            de=input("Введите номер брони для её отмены:")
            if not de.isdigit():
                print("Введите число")
                continue
            de = int(de)
            if 1 <= de <= len(user_bookings):
                booking_to_delete = user_bookings[de - 1]

self.available_slots.append(booking_to_delete.datetime)
                self.available_slots.sort()
                self.bookings.remove(booking_to_delete)
                print(f"Бронирование отменено")
                break
            else:
                print("Бронь не найдена")
    def change_booking(self):
        user_bookings = [b for b in self.bookings if b.user_id ==
self.current_user.id]
        if not user_bookings:
            print("У вас нет активных бронирований")
            return
        while True:
            ism = input("Введите номер брони для её изменения:")
            if not ism.isdigit():
                print("Введите число")
                continue
            ism = int(ism)
            if 1 <= ism <= len(user_bookings):
```

```
                booking_to_change = user_bookings[ism - 1]
                quest = None
                for q in all_quests:
                    if q.id == booking_to_change.quest_id:
                        quest = q
                        break
                if not quest:
                    print("Квест не найден")
                    return
                while True:
                    player_count = input("Введите новое
количество игроков: ")
                    if player_count.isdigit():
                        player_count = int(player_count)
                        if quest.min_players <= player_count <=
quest.max_players:
                            booking_to_change.player_count =
player_count
                            print(f"Количество игроков изменено
на {player_count}")
                            break
                        print("Неверное количество игроков.}")
                    else:
                        print("Введите число")
                break
            else:
                print("Неверный номер брони")
    def review(self):
        while True:
            print("Мои отзывы")
            print("1. Просмотр отзывов")
            print("2. Редактировать отзыв")
            print("3. Удалить отзыв")
            print("4.Назад")
            choice = input("Выбор: ")

            if choice == "1":
                self.display_review()
            elif choice == "2":
                self.change_review()
            elif choice=="3":
                self.del_review()
            elif choice == "4":
                self.current_user = None
                print("Вы вышли")
                break
    def display_review(self):
        my_reviews_list = [r for r in self.reviews if r.user_id
== self.current_user.id]
```

```
        if not my_reviews_list:
            print("У вас пока нет отзывов")
            return
        for i, review in enumerate(my_reviews_list, 1):
            quest = None
            for q in all_quests:
                if q.id == review.quest_id:
                    quest = q
                    break
            quest_name = quest.name
            print(f"Квест: {quest_name}")
            print(f"Номер отзыва: {review.id}")
            print(f"   Общее впечатление: {review.overall}/5")
            print(f"   Актерская игра: {review.acting}/5")
            print(f"   Логичность заданий: {review.logic}/5")
            print(f"   Отзыв: {review.text}")
    def change_review(self):
        my_reviews_list = [r for r in self.reviews if r.user_id
== self.current_user.id]
        if not my_reviews_list:
            print("У вас пока нет отзывов")
            return
        while True:
            choice = input("Введите номер отзыва для изменения:
")
            if not choice.isdigit():
                print("Введите число")
                continue
            choice = int(choice)
            if 1 <= choice <= len(my_reviews_list):
                review_to_change = my_reviews_list[choice - 1]
                while True:
                    new_text = input("Введите новый текст отзыва:
")
                    if len(new_text) < 10:
                        print("Отзыв слишком короткий (минимум 10
символов)")
                        continue
                    elif len(new_text) > 500:
                        print("Отзыв слишком длинный (максимум
500 символов)")
                        continue
                    else:
                        review_to_change.text = new_text
                        print("Отзыв успешно изменен!")
                        break
                break
            else:
                print("Неверный номер")
```

```
    def del_review(self):
        my_reviews_list = [r for r in self.reviews if r.user_id
== self.current_user.id]
        if not my_reviews_list:
            print("У вас пока нет отзывов")
            return
        while True:
            choice = input("Введите номер отзыва для удаления: ")
            if not choice.isdigit():
                print("Введите число")
                continue
            choice = int(choice)
            if 1 <= choice <= len(my_reviews_list):
                review_to_delete = my_reviews_list[choice - 1]
                confirm = input(f"Вы уверены, что хотите удалить
отзыв о квесте? (да/нет): ")
                if confirm.lower() == "да":
                    self.reviews.remove(review_to_delete)
                    print("Отзыв успешно удален!")
                else:
                    print("Удаление отменено")
                return
            else:
                print("Неверный номер")
```

2. Листинг файла console.py:

```
class ConsoleIO:
    def input(self, field, defvalue=None):
        return input(f"{field}: ")
    def output(self, title, field):
        print(f"{title}: {field}")
```

3. Листинг файла main.py:

```
from book import Book
def main():
    book = Book()
    while True:
        if book.current_user is None:
            print("1. Создать пользователя")
            print("2. Удалить пользователя")
            print("3. Изменить пользователя")
            print("4. Показать всех пользователей")
            print("5. Войти в пользователя")
            print("6. Сохранить в файл")
            print("7. Загрузить из файла")
            choice = input("Выбор: ")
            if choice == "1":
                book.add()
            elif choice == "2":
```

```
                book.delete()
            elif choice == "3":
                book.change()
            elif choice == "4":
                book.write()
            elif choice == "5":
                book.login()
            elif choice == "6":
                book.store()
            elif choice == "7":
                book.load()
        else:
                book.user_menu()
if __name__ == "__main__":
    main()
```

4. Листинг файла models.py:

```
from dataclasses import dataclass
import datetime
from console import ConsoleIO
from typing import List, Dict
@dataclass
class Quest:
    id: int
    name: str
    duration: int  # длительность в минутах
    min_players: int
    max_players: int
all_quests: List[Quest] = [
    Quest(1, "Слепые", 90, 2, 8),
    Quest(2, "Живая коллекция", 90, 4, 6),
    Quest(3, "Карантин", 60, 2, 8),
]
@dataclass
class User:
    id: int = 0
    first_name: str = ""
    second_name: str = ""
    phone: str = ""
    password: str = ""
    confirm_password: str = ""
    email: str = ""
    time = 0

    def validate_name(self, value):
        if not value:
            print("Поле обязательно для заполнения")
            return False
        if len(value) < 2:
            print("Имя должно содержать минимум 2 символа")
```

```
            return False
        if any(char in "1234567890" for char in value):
            print("Имя не должно содержать цифры")
            return False
        if any(char in "!@#$%^&*()_+=[]{}|;':,./<>?~" for char in
value):
            print("Имя не должно содержать спецсимволы")
            return False
        return True
    def validate_second_name(self,value):
        if not value:
            print("Поле обязательно для заполнения")
            return False
        if len(value) < 2:
            print("Фамилия должна содержать минимум 2 символа")
            return False
        if any(char in "!@#$%^&*()_+=[]{}|;':,./<>?~" for char in
value):
            print("Фамилия не должна содержать спецсимволы")
            return False
        if any(char in "1234567890" for char in value):
            print("Фамилия не должна содержать цифры")
            return False
        return True
    def validate_phone(self,value):
        if not value:
            print("Поле обязательно для заполнения")
            return False
        if any(char in "!@#$%^&*()_-+=[]{}|;':,./<>?~" for char
in value):
            print("Номер телефона не должен содержать
спецсимволы")
            return False
        if any(
                char in
"АБВГДЕЁЖЗИЙКЛМНОПРСТУФХЦЧШЩЪЫЬЭЮЯабвгдеёжзийклмнопрстуфхцчшщъыьэ
юяABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"
                for char in value):
            print("Номер телефона должен содержать только цифры")
            return False
        if (value[0] != '7'):
            print("Номер телефона должен начинаться с 7")
            return False
        if len(value) != 11:
            print("Номер телефон должен содержать ровно 11 цифр")
            return False
        return True
    def validate_password(self,value):
        if not value:
```

```
            print("Поле обязательно для заполнения")
            return False
        elif len(value) < 8:
            print("Пароль должен содержать не менее 8 символов")
            return False
        elif len(value) > 16:
            print("Пароль должен содержать не более 16 символов")
            return False
        elif not any(char in
"ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz" for char
in value):
            print("Пароль должен содержать хотя бы одну латинскую
букву")
            return False
        return True
    def validate_email(self, value):
        if not value:
            print("Email пропущен")
            return True
        if "@" not in value:
            print("Email должен содержать символ @")
            return False
        if "." not in value:
            print("Email должен содержать точку (.)")
            return False
        if len(value) < 5:
            print("Email слишком короткий")
            return False
        return True
    def __post_init__(self):
        self.time = datetime.datetime.now()
        self.io = ConsoleIO()
    def change(self):
        while True:
            print(f"1. Имя: {self.first_name}")
            print(f"2. Фамилия: {self.second_name}")
            print(f"3. Телефон: {self.phone}")
            print(f"4. Email: {self.email}")
            print(f"5. Пароль:  {self.password}")
            print("0. Завершить редактирование")
            choice = input("Выберите поле для изменения: ")
            if choice == "0":
                print("Редактирование завершено")
                break
            elif choice == "1":
                while True:
                    new_value = input("Новое имя: ")
                    if new_value:
                        if self.validate_name(new_value):
```

```
                            self.first_name = new_value
                            print("Имя изменено")
                            break
            elif choice == "2":
                while True:
                    new_value = input("Новая фамилия: ")
                    if new_value:
                        if self.validate_second_name(new_value):
                            self.second_name = new_value
                            print("Фамилия изменена")
                            break
            elif choice == "3":
                while True:
                    new_value = input("Новый номер телефона: ")
                    if new_value:
                        if self.validate_phone(new_value):
                            self.phone = "+" + new_value
                            print("Телефон изменён")
                            break
            elif choice == "4":
                new_value = input(f"Новый email (Enter —
удалить): ")
                if self.validate_email(new_value):
                    self.email = new_value
                    print("Email изменён" if new_value else
"Email удалён")
            elif choice == "5":
                while True:
                    new_value = input("Новый пароль: ")
                    if new_value:
                        if self.validate_password(new_value):
                            confirm = input("Подтвердите пароль:
")
                            if confirm == new_value:
                                self.password = new_value
                                self.confirm_password = confirm
                                print("Пароль изменён")
                            else:
                                print("Пароли не совпадают")
            else:
                print("Неверный номер, попробуйте снова")

    def change_profile(self):
        print("Редактирование профиля")
        while True:
            print(f"1. Имя:      {self.first_name}")
            print(f"2. Фамилия:  {self.second_name}")
            print(f"3. Email:    {self.email}")
            print(f"4. Пароль:     {self.password}")
```

```
            print("0. Назад")
            choice = input("Выбор: ")
            if choice == "0":
                print("Редактирование завершено")
                break
            elif choice == "1":
                while True:
                    new_value = input("Новое имя: ")
                    if new_value:
                        if self.validate_name(new_value):
                            self.first_name = new_value
                            print("Имя изменено")
                            break
            elif choice == "2":
                while True:
                    new_value = input("Новая фамилия: ")
                    if new_value:
                        if self.validate_second_name(new_value):
                            self.second_name = new_value
                            print("Фамилия изменена")
                            break
            elif choice == "3":
                new_value = input(f"Новый email (Enter —
удалить): ")
                if self.validate_email(new_value):
                    self.email = new_value
                    print("Email изменён" if new_value else
"Email удалён")
            elif choice == "4":
                while True:
                    new_value = input("Новый пароль: ")
                    if new_value:
                        if self.validate_password(new_value):
                            confirm = input("Подтвердите пароль:
")
                            if confirm == new_value:
                                self.password = new_value
                                self.confirm_password = confirm
                                print("Пароль изменён")
                            else:
                                print("Пароли не совпадают")
            else:
                print("Неверный номер, попробуйте снова")

    def read(self):
        while True:
            value = input('Имя: ')
            if self.validate_name(value):
                self.first_name = value
```

```
                break
        while True:
            value = input('Фамилия: ')
            if self.validate_second_name(value):
                self.second_name = value
                break
        while True:
            value = input('Номер телефона: ')
            if self.validate_phone(value):
                self.phone = "+" + value
                break
        while True:
            value = input('Email (необязательно, Enter —
пропустить): ')
            if self.validate_email(value):
                self.email = value
                break
        while True:
            value = input('Пароль ')
            if self.validate_password(value):
                self.password = value
                break
        while True:
            confirm_password = self.io.input('Повторите пароль ')
            if confirm_password != self.password:
                print("Пароли не совпадают")
                continue
            self.confirm_password = confirm_password
            break
        while True:
            confirm_password=self.io.input('Повторите пароль ')
            if confirm_password !=self.password:
                print("Пароли не совпадают")
                continue
            self.confirm_password=confirm_password
            break
    def write(self):
        self.io.output('ID', self.id)
        self.io.output('Имя', self.first_name)
        self.io.output('Фамилия', self.second_name)
        self.io.output('Номер телефона', self.phone)
        self.io.output('Email', self.email or 'не указан')
        self.io.output('Пароль', self.password)
        self.io.output('Time', self.time)
@dataclass
class Booking:
    id: int = 0
    user_id: int = 0
    quest_id: int = 0
```

```
    datetime: str = ""
    player_count: int = 1
@dataclass
class Review:
    id: int = 0
    user_id: int = 0
    quest_id: int = 0
    logic: int=5
    overall: int=5
    acting: int = 5
    text: str = ""
```

5. Листинг файла storage.py:

```
import pickle
class PickleStorage:
    def __init__(self, book):
        self.book = book
    def store(self):
        pickle.dump((self.book.maxID, self.book.items),
open("data.dat", "wb"))
    def load(self):
        (self.book.maxID, self.book.items) =
pickle.load(open("data.dat", "rb"))
```

Вывод:

В ходе выполнения лабораторной работы были освоены базовые принципы объектно-ориентированного программирования на языке Python. Были реализованы:

- Классы для описания сущностей предметной области.
- Контейнер для управления коллекцией объектов.
- Стратегия сохранения данных с использованием модуля pickle.
- Консольный интерфейс с системой меню.

Полученное приложение позволяет вести учет пользователей и заказов, обеспечивает сохранность данных между сеансами работы и проверяет вводимую пользователем информацию на корректность.